

Day 2 Live-Coding Session — 5 Problems

1. Vowel & Consonant Counter

Scenario

A library orientation kiosk counts vowels and consonants in a submitted book title for a simple text-stats display.

Task

- Accept a string (assume only letters and spaces).
- Loop through each character using `charAt()`.
- Count vowels (a, e, i, o, u — case-insensitive) and consonants separately; ignore spaces.
- Print the total vowels and total consonants.

Suggested method signature(s)

```
void countVowelsAndConsonants(String text)
```

Sample Input / Output

Input	Output
"Java Programming"	Vowels: 5 Consonants: 10

Concepts covered: `charAt()`, `length()`, loops, case-insensitive character comparison.

2. CSV Student Record Parser

Scenario

The T&P team receives student registration data as CSV lines and needs a quick parser to split each line into fields and print a formatted record.

Task

- Accept a CSV line in the form "Name,RollNumber,Department".
- Use `split(",")` to break it into fields.
- Validate that exactly 3 fields are present; if not, print "Invalid Record".
- Print a formatted record: "Name: ... | Roll No: ... | Dept: ...".

Suggested method signature(s)

```
void parseStudentRecord(String csvLine)
```

Sample Input / Output

Input	Output
"Ananya Verma,RA2211003010123,CSE"	Name: Ananya Verma Roll No: RA2211003010123 Dept: CSE
"Ananya Verma,CSE"	Invalid Record

Concepts covered: *split(), array length validation, string concatenation, formatted output.*

3. File Extension Validator

Scenario

An assignment-upload portal must check whether an uploaded filename has an accepted extension (pdf, docx, zip) regardless of case, before accepting the submission.

Task

- Accept a filename string.
- Find the last '.' using `lastIndexOf('.')` and extract the extension with `substring()`.
- Compare the extension case-insensitively against the accepted list: pdf, docx, zip.
- Print "Accepted" or "Rejected — invalid file type".

Suggested method signature(s)

```
String validateFileExtension(String filename)
```

Sample Input / Output

Input	Output
"Assignment1.PDF"	Accepted
"notes.txt"	Rejected — invalid file type

Concepts covered: *lastIndexOf(), substring(), equalsIgnoreCase(), conditional logic.*

4. Masked Phone Number Formatter

Scenario

A student-support call center displays a partially masked version of a registered phone number on-screen for privacy, while agents confirm identity using the last 4 digits.

Task

- Accept a phone number as a string.
- Validate that it is exactly 10 digits (all numeric).
- Build a masked version showing "XXXXXX" followed by the last 4 digits, using `StringBuilder`.
- Insert a "-" between the mask and the last 4 digits for readability.
- Print the final masked number, or an error message if validation fails.

Suggested method signature(s)

```
String maskPhoneNumber(String phone)
```

Sample Input / Output

Input	Output
"9876543210"	XXXXXX-3210
"98765"	Invalid phone number

Concepts covered: String length/digit validation, substring(), StringBuilder insert(), masking patterns.

5. Bank Transaction Reference Generator & Validator

Scenario

A fintech onboarding module for a placement-prep hackathon needs to both normalize and validate transaction reference codes. A valid reference is exactly 14 characters: 3 letters (bank code) + 6 digits (date, ddMMyy) + 5 digits (sequence number). Users sometimes paste codes with stray spaces or a mixed-case bank code.

Task

- Accept a raw reference string that may contain leading/trailing spaces.
- Normalize it: trim() the spaces, then uppercase only the first 3 characters using substring() + concatenation — leave the rest untouched.
- Validate: exactly 14 characters after normalization; the first 3 characters are letters; the remaining 11 are digits (use Character.isLetter() / isDigit() in a loop — no regex).
- If valid, build a formatted display line with StringBuilder: "[BANKCODE] DATE: dd/MM/yy | SEQ: 12345".
- If invalid, print the specific reason: wrong length, non-letter bank code, or non-digit body.

Suggested method signature(s)

```
String normalizeReference(String raw) + String validateAndFormat(String reference)
```

Sample Input / Output

Input	Output
" hdf03022600042 "	[HDF] DATE: 03/02/26 SEQ: 00042
"12F03022600042"	Invalid: bank code must be 3 letters

Concepts covered: trim(), substring(), string concatenation, Character.isLetter()/isDigit(), StringBuilder, multi-stage validation.